JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.invoke

Class CallSite

java.lang.Object
    java.lang.invoke.CallSite

Direct Known Subclasses:

ConstantCallSite, MutableCallSite, VolatileCallSite

```
public abstract class CallSite
extends Object
```

A `CallSite` is a holder for a variable `MethodHandle`, which is called its `target`. An `invokedynamic` instruction linked to a `CallSite` delegates all calls to the site's current target. A `CallSite` may be associated with several `invokedynamic` instructions, or it may be "free floating", associated with none. In any case, it may be invoked through an associated method handle called its dynamic invoker.

`CallSite` is an abstract class which does not allow direct subclassing by users. It has three immediate, concrete subclasses that may be either instantiated or subclassed.

- If a mutable target is not required, an `invokedynamic` instruction may be permanently bound by means of a constant call site.
- If a mutable target is required which has volatile variable semantics, because updates to the target must be immediately and reliably witnessed by other threads, a volatile call site may be used.
- Otherwise, if a mutable target is required, a mutable call site may be used.

A non-constant call site may be *relinked* by changing its target. The new target must have the same type as the previous target. Thus, though a call site can be relinked to a series of successive targets, it cannot change its type.

Here is a sample use of call sites and bootstrap methods which links every dynamic call site to print its arguments:

```
static void test() throws Throwable {
    // THE FOLLOWING LINE IS PSEUDOCODE FOR A JVM INSTRUCTION
    InvokeDynamic[#bootstrapDynamic].baz("baz arg", 2, 3.14);
}
private static void printArgs(Object... args) {
  System.out.println(java.util.Arrays.deepToString(args));
}
private static final MethodHandle printArgs;
static {
  MethodHandles.Lookup lookup = MethodHandles.lookup();
  Class thisClass = lookup.lookupClass();  // (who am I?)
  printArgs = lookup.findStatic(thisClass,
      "printArgs", MethodType.methodType(void.class, Object[].class));
}
private static CallSite bootstrapDynamic(MethodHandles.Lookup caller, String n
  // ignore caller and name, but match the type:
  return new ConstantCallSite(printArgs.asType(type));
}
```

Method Summary

Concrete Methods **Abstract Methods**

Modifier and Type	Method	Description
abstract **MethodHandle**	**dynamicInvoker**()	Produces a method handle equivalent to an invokedynamic instruction which has been linked to this call site.
abstract **MethodHandle**	**getTarget**()	Returns the target method of the call site, according to the behavior defined by this call site's specific class.
abstract void	**setTarget**(**MethodHandle** newTarget)	Updates the target method of this call site, according

		to the behavior defined by this call site's specific class.
MethodType	**type**()	Returns the type of this call site's target.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

type

```
public MethodType type()
```

Returns the type of this call site's target. Although targets may change, any call site's type is permanent, and can never change to an unequal type. The `setTarget` method enforces this invariant by refusing any new target that does not have the previous target's type.

Returns:

the type of the current target, which is also the type of any future target

getTarget

```
public abstract MethodHandle getTarget()
```

Returns the target method of the call site, according to the behavior defined by this call site's specific class. The immediate subclasses of `CallSite` document the class-specific behaviors of this method.

Returns:

the current linkage state of the call site, its target method handle

See Also:

ConstantCallSite, VolatileCallSite, setTarget(java.lang.invoke.MethodHandle),

ConstantCallSite.getTarget(), MutableCallSite.getTarget(), VolatileCallSite.getTarget()

setTarget

```
public abstract void setTarget(MethodHandle newTarget)
```

Updates the target method of this call site, according to the behavior defined by this call site's specific class. The immediate subclasses of CallSite document the class-specific behaviors of this method.

The type of the new target must be equal to the type of the old target.

Parameters:

newTarget - the new target

Throws:

NullPointerException - if the proposed new target is null

WrongMethodTypeException - if the proposed new target has a method type that differs from the previous target

See Also:

getTarget(), ConstantCallSite.setTarget(java.lang.invoke.MethodHandle), MutableCallSite.setTarget(java.lang.invoke.MethodHandle), VolatileCallSite.setTarget(java.lang.invoke.MethodHandle)

dynamicInvoker

```
public abstract MethodHandle dynamicInvoker()
```

Produces a method handle equivalent to an invokedynamic instruction which has been linked to this call site.

This method is equivalent to the following code:

```
MethodHandle getTarget, invoker, result;
getTarget = MethodHandles.publicLookup().bind(this, "getTarget", MethodType
invoker = MethodHandles.exactInvoker(this.type());
result = MethodHandles.foldArguments(invoker, getTarget)
```

Returns:

a method handle which always invokes this call site's current target

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.